Assignment 6 - Visitor Pattern

Student: Timur Naumov
Group: SE-2537
Course: Software Design Patterns
Project: Advanced File System

GitHub: tnaumovv/assignment6-visitor-pattern

1. Purpose and structure

This project models files and directories in memory. It does not read or change real files. The same tree supports three operations: total byte size, file search and statistics. These operations are separate visitors, so the node classes do not contain the calculation or search rules.

`Element` declares `accept(Visitor)` and `getName()`. `AbstractNode` implements the shared name, parent and relative-path/depth helpers. `FileNode` stores a non-negative byte size. `DirectoryNode` stores children. Both concrete nodes implement `accept` with `visitor.visit(this)`.

`Visitor` declares two overloaded `visit` methods and `reset(root)`. `TotalSizeVisitor` sums file sizes. `SearchVisitor` combines a filename predicate with `size >= minimumSize`. `StatisticsVisitor` counts files, directories and maximum depth. `Main` creates the example tree and prints the results.

2. Double dispatch and traversal

The first dispatch selects the element's `accept` method. For a file, Java enters `FileNode.accept`. Inside that method, `this` has type `FileNode`, so the compiler selects the `visit(FileNode)` overload. Runtime dispatch then selects that method's implementation on the actual visitor. Overloading itself is resolved at compile time, not runtime. [1]

`TreeWalker` owns traversal. It also implements `Visitor` as a small adapter: each popped node calls `accept(this)`, then the matching walker overload calls `operation.visit(node)`. Only the directory overload pushes children. The operation visitors never walk the tree, so each reachable node reaches the operation once.

The walker uses a stack for depth-first traversal. Children are pushed in reverse insertion order. This avoids recursive traversal and works with empty and deeply nested directories. Before every run, the walker calls `reset(root)` on the operation. The same visitor can be reused sequentially. Its results describe its latest completed run; a failed run must be rerun before using its result.

3. Integrity, paths and depth

Names cannot be blank, `.` or `..`, or contain `/` or `\`. Names and sizes cannot change after construction. `DirectoryNode.add` rejects a second parent, duplicate sibling names and cycles before changing the tree. A file and directory cannot share a sibling name. Invalid values/mutations raise `IllegalArgumentException`; null arguments raise `NullPointerException`.

Children are returned as an unmodifiable snapshot. Parent changes are package-controlled. The API supports adding nodes, but not removal or moving. Operations run sequentially and the tree is not changed during a walk. Total-size overflow raises `ArithmeticException` instead of producing an incorrect negative total.

Paths are built by following parents up to the chosen root. A directory root's name is excluded; a root file is returned under its own name. Paths use `/` and are sorted lexicographically. Search is case-sensitive in the demo. Depth counts parent links to the chosen root: the root is 0, its children are 1, and so on. Empty directories also contribute to maximum depth.

4. Example data and actual output

The example in `Main.createTree()` is:

```
coursework/
  readme.txt             120 bytes
  docs/
    notes.txt            80 bytes
    report.txt           350 bytes
    drafts/
      plan.txt           100 bytes
  images/
    logo.png             900 bytes
  empty/
```

Command: `mvn compile exec:java`

```
Root: coursework
Total size: 1550 bytes
Search: suffix=.txt, minimum=100 bytes
Matches: [docs/drafts/plan.txt, docs/report.txt, readme.txt]
Files: 5
Directories: 5
Maximum depth: 3
Repeated run unchanged: true
```

The total is `120 + 80 + 350 + 100 + 900 = 1550`. The root counts as a directory. `plan.txt` passes the inclusive threshold because its size is exactly 100. `notes.txt` fails the size condition, and `logo.png` fails the filename condition.

Changing the configuration to `mvn compile exec:java -Dexec.args=".png 500"` returns `Matches: [images/logo.png]`. Size and statistics still describe the whole tree; the search filters do not filter the other visitors.

5. Five Clean Code practices

1. **Meaningful naming:** `getTotalSize`, `minimumSize` and `getMaxDepth` explain the values without additional comments.

2. **Single Responsibility:** `TreeWalker` controls traversal; `SearchVisitor.visit(FileNode)` only applies the search conditions and records a path.

3. **Small methods:** both element `accept` methods contain one call, making the dispatch step easy to find and trace.

4. **Encapsulation:** `DirectoryNode.getChildren()` returns `List.copyOf(children)`, so callers cannot bypass `add` and break ownership rules.

5. **No type-checking chains:** `FileNode.accept` and `DirectoryNode.accept` select the correct overload through `visitor.visit(this)`; no casts, reflection or runtime type checks select node behavior.

6. Automated verification

Command: `mvn test`. The suite was run with JDK 17.0.21. Maven reported:

```
Tests run: 22, Failures: 0, Errors: 0, Skipped: 0
BUILD SUCCESS
```

- `VisitorTest` (10 tests): correct overloads and one visit per node; nested size; combined filename/size filters; sorted root-relative paths; a selected nested root; empty/no-match cases; repeated runs after additions; a root file of size zero; invalid search configuration; total-size overflow and recovery.

- `TreeIntegrityTest` (7 tests): negative and zero sizes; invalid names; duplicate file/directory sibling names; multiple parents; self/ancestor cycles; unmodifiable children and null rejection; valid nesting 1,500 directories deep.

- `StatisticsVisitorTest` (5 tests): all three statistics on a nested tree; an empty directory root; a file root; resetting all counters and counting empty-directory depth; depth relative to a selected subtree.

These tests cover checkpoints T1-T8. Fixtures and expected values are defined in the test classes, separately from the demonstration data.

7. Extension and trade-offs

The first implementation commit contains the elements, the full `Visitor` contract, `TreeWalker`, `TotalSizeVisitor` and `SearchVisitor`. The next commit adds `StatisticsVisitor` and its tests. The existing elements, their `accept` methods, the interface and the walker are unchanged in that extension.

Adding another operation is straightforward: implement `Visitor`, reset its state and handle both element types. The trade-off is adding a new element type. For example, a `LinkNode` would require a new `visit(LinkNode)` method, a matching `accept`, and updates to existing visitor implementations, including the walker. Visitor works best when element types are stable and operations change more often.

Keeping traversal in one walker avoids repeated walking code, but introduces an adapter and an explicit reset step. Paths and depth are calculated from parent links, so their cost grows with nesting depth. The model uses an ordinary bidirectional parent/children association: exclusive single-parent membership is checked by `add`, while deletion/lifetime ownership is not modeled.

8. UML notation

The diagram on the next page shows both interfaces, the shared abstract node, both concrete elements, the walker and all three operation visitors. `Main`, tests and standard library types are omitted. Member signatures and typed fields show dependencies/references that do not need duplicate arrows.

A dashed line with a hollow triangle means interface realization; a solid line with a hollow triangle means class inheritance. Both triangles point to the parent type. `AbstractNode` realizes `Element`; the two nodes inherit that interface through the base class. A dashed open arrow from `Element` to `Visitor` shows its `accept` parameter dependency. Solid open arrows show navigable associations. [2]

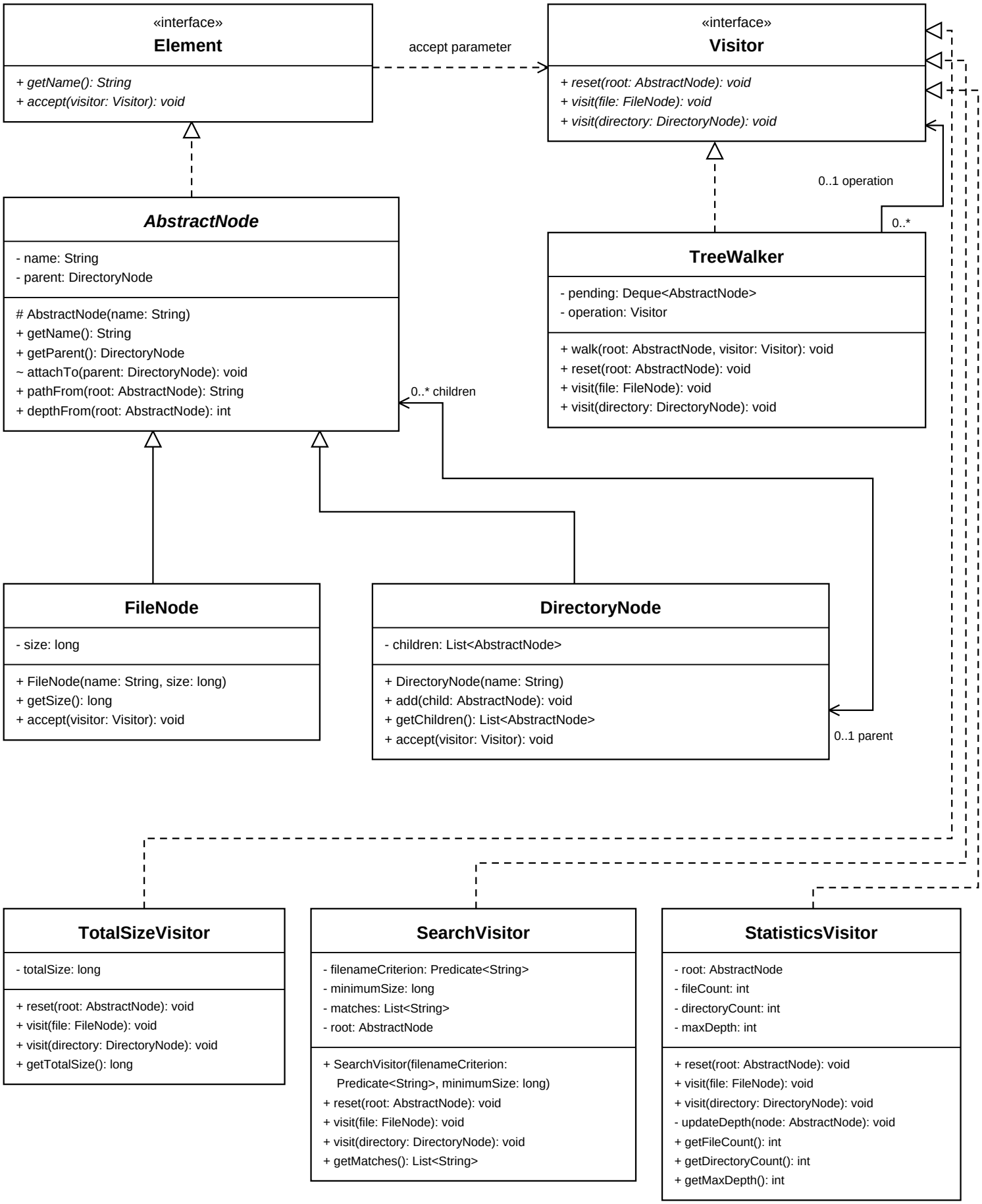
The parent/children association is navigable both ways: each node has `0..1` parent directory, and each directory has `0..*` children. The walker's operation association has `0..1` at the `Visitor` end because the field is null while idle. + means public, - private, # protected and ~ package access. The abstract class name and interface operations are italic. All relationship lines use the same width; their meaning comes from the line style and arrowhead. No diamond or lollipop notation is needed here.

References

[1] Oracle, Java Language Specification 17, sections 15.12.2 and 15.12.4.4: https://docs.oracle.com/javase/specs/jls/se17/html/jls-15.html

[2] OMG, UML 2.5.1, class, interface and relationship notation: https://www.omg.org/spec/UML/2.5.1

Visitor Pattern - UML Class Diagram



Dashed + hollow triangle: realizes interface. Solid + hollow triangle: inherits class.
Solid open arrows: association. Dashed open arrow: dependency. Fields also show references.